PARTNER MAINNET DEPLOYMENT WRAPPER DESIGN

Date: 2026-03-23
Repo: ‘sig-net/mpc-infrastructure‘
Scope: design proposal for a simpler GCP-only partner deployment flow for mainnet nodes
Tool name: ‘mpc-infra‘

EXECUTIVE SUMMARY

The current partner deployment flow works, but it asks partner operators to understand too much infrastructure detail:

? Terraform state setup
? GCP project prep
? Secret naming and Secret Manager population
? per-node tfvars layout
? VM and load balancer details
? rollout and verification steps

That is survivable for internal operators, but it creates friction for partners who are already prone to getting lost in large or highly technical setup flows.

The recommended direction is **not** to replace the existing Terraform immediately. Instead, build a thin, opinionated **CLI wrapper** called ‘mpc-infra‘ around the current ‘terraform/partner-mainnet‘ deployment so partners interact with a guided workflow while the chain signatures team keeps control of the underlying deployment details.

GOALS

1. Keep **GCP as the only supported provider**.
2. Reduce partner cognitive load.
3. Preserve the ability for the chain signatures team to evolve deployment internals.
4. Keep the partner-facing interface stable even if Terraform internals change.
5. Catch mistakes early with strong preflight validation.
6. Avoid forcing partners to learn Kubernetes.

NON-GOALS

? Multi-cloud support.
? A full replacement of Terraform in phase 1.
? A Kubernetes-based partner operating model.

? Arbitrary infrastructure customization by partners.

CURRENT PAIN POINTS

1. Too many infrastructure concepts are exposed

Partners currently have to reason about:

? project and API setup
? Terraform backend bucket creation
? ‘tfvars‘ editing
? many secret IDs
? DNS planning
? image pinning
? node count and node config structure

That is a lot of surface area for operators who are not living in this codebase.

2. Validation is late

Many mistakes are only discovered during or after Terraform operations, for example:

? missing Secret Manager secrets
? secret name typos
? wrong project ID
? APIs not enabled
? missing IAM permissions
? bad DNS assumptions

3. The partner-facing config is shaped like the implementation

The ‘node_configs‘ object is useful for Terraform, but it is not the simplest mental model for a partner operator.

4. Internal evolution is coupled to partner-facing docs

As the app changes, the team needs to add secrets, env vars, and deployment rules. That is normal. The problem is that the partner deployment interface changes too directly and too often.

RECOMMENDED APPROACH

Build a **partner deployment wrapper CLI** that sits in front of the existing Terraform module.

Core idea

Partners should describe **node intent**, not Terraform structure.

The wrapper should:

1. ask for a small set of required inputs
2. write a normalized config file
3. validate the GCP environment and required secrets
4. generate the Terraform inputs deterministically
5. run safe Terraform subcommands in the correct order
6. print a short, guided next-step summary

This gives partners one paved road while letting the team keep Terraform as the implementation layer.

PROPOSED USER EXPERIENCE

Partner workflow

A partner should be able to do something like:

mpc-infra init
mpc-infra validate
mpc-infra plan
mpc-infra deploy
mpc-infra status
mpc-infra upgrade

What each command should do

‘mpc-infra init‘

Guided setup:

? choose or confirm deployment type: ‘mainnet‘
? enter GCP project ID
? choose region and zone from supported defaults
? enter hostname/domain
? enter NEAR account ID
? enter node count if multi-node is still needed
? enter or confirm required secret names
? write a simple partner config file

‘mpc-infra validate‘

Preflight checks before any Terraform planning:

? ‘gcloud‘ auth present
? target project exists
? required APIs enabled
? state bucket exists and is reachable
? required Secret Manager secrets exist
? required IAM roles are sufficient
? required DNS fields are populated
? config is internally consistent

This should fail early with human-readable messages.

‘mpc-infra plan‘

? generate Terraform input from the partner config
? run Terraform plan using the generated inputs
? summarize creates/changes in plain English

‘mpc-infra deploy‘

? require a successful validate step first
? optionally require confirmation if the plan changed since the last run
? run the apply path under the wrapper
? print the resulting IPs, DNS actions, and verification steps

‘mpc-infra status‘

? show deployment metadata
? show instance names and load balancer IPs
? show expected DNS targets
? show a short verification checklist

‘mpc-infra upgrade‘

? detect the currently deployed image tag
? resolve the target image tag from the latest published release by default
? support an explicit target with ‘--tag <tag>‘
? tell the operator when no upgrade is needed
? if an upgrade is needed, update the deployment input and safely restart/redeploy the node
? print the resulting version and post-upgrade verification guidance

Recommended later flags:

? ‘--check-only‘
? ‘--dry-run‘
? ‘--yes‘

STABLE PARTNER-FACING CONFIG

The wrapper should write a small config file that is easier to understand than raw Terraform variables.

Example:

```
version: 1
profile: mainnet
project_id: partner-multichain
region: europe-west1
zone: europe-west1-b
state_bucket: multichain-terraform-partner
node_count: 1
nodes:
  - account_id: company.near
    domain: mpc.company.com
    secrets:
      account_sk: multichain-account-sk-mainnet-0
      cipher_sk: multichain-cipher-sk-mainnet-0
      sign_sk: multichain-sign-sk-mainnet-0
```

sk_share: multichain-sk-share-mainnet-0
eth_account_sk: multichain-eth-account-sk-mainnet-0
eth_consensus_rpc: multichain-eth-consensus-rpc-url-mainnet
eth_execution_rpc: multichain-eth-execution-rpc-url-mainnet
sol_account_sk: multichain-sol-account-sk-mainnet-0
sol_rpc_http: multichain-sol-rpc-http-url-mainnet
sol_rpc_ws: multichain-sol-rpc-ws-url-mainnet
hydration_rpc_ws: multichain-hydration-rpc-ws-url-mainnet
hydration_signer_uri: multichain-hydration-signer-uri-mainnet

The wrapper can translate that into the current ‘terraform/partner-mainnet‘ input shape.

WHY A WRAPPER IS A BETTER FIT THAN EXPOSING MORE TERRAFORM

1. It reduces partner error rate

A wrapper can catch bad input before Terraform touches anything.

2. It preserves your internal control

The team can keep changing:

? image tags
? startup scripts
? env wiring
? health checks
? logging defaults
? additional secret requirements

without making partners absorb every infrastructure detail directly.

3. It supports gradual migration

You do not need to rewrite the deployment stack all at once.

Phase 1 can keep the current Terraform intact and just wrap it.

4. It matches your support reality

If partners are easily confused, reducing visible moving parts matters more than maximizing theoretical flexibility.

RECOMMENDED ARCHITECTURE

Layer 1: partner config

A small YAML file committed or stored locally by the partner.

Layer 2: validation engine

Checks local tooling, GCP access, APIs, IAM, secrets, and config completeness.

Layer 3: Terraform renderer

Generates the exact Terraform variables or ‘.auto.tfvars.json‘ file expected by ‘terraform/partner-mainnet‘.

Layer 4: Terraform runner

Runs Terraform in a controlled, guided way with consistent commands and output.

Layer 5: post-deploy guide

Prints:

? IPs
? DNS records to create
? verification commands
? where to find logs
? rollback/update guidance

OPINIONATED DEFAULTS

To reduce support burden, the wrapper should make more decisions automatically.

Recommended defaults to hide from partners unless there is a strong reason not to:

? network = ‘default‘
? subnetwork = ‘default‘
? region default = ‘europe-west1‘
? standard supported zone set
? default image source unless explicitly overridden by the team
? standard machine type
? default health check port/path
? standard logging and monitoring wiring
? standard secret naming suggestions

The more defaults that are team-owned, the fewer places partners can drift into unsupported states.

VALIDATION RULES TO PRIORITIZE

These matter the most in a first implementation:

1. GCP auth is active and points at the intended project.
2. Secret Manager API is enabled.
3. Compute Engine API is enabled.
4. Terraform state bucket exists.
5. Every referenced secret exists.
6. Every required config field is present.
7. Domain names are syntactically valid.
8. Node count matches the number of node entries.
9. The deployment image/tag is set to a supported value.
10. Required app feature secrets are present for the selected release profile.

RELEASE AND CHANGE-MANAGEMENT MODEL

The wrapper should support a versioned schema and a versioned deployment profile.

That means:

? wrapper config schema version, for example ‘version: 1‘
? deployment profile version or release channel controlled by the team
? validation rules that can tell partners what changed between releases

This matters because the node app will continue to evolve.

MIGRATION STRATEGY

Phase 1: wrapper-only, no Terraform redesign

? keep current ‘terraform/partner-mainnet‘
? add the ‘mpc-infra‘ CLI and config schema
? add validation and generated tfvars
? update docs to tell partners to use the wrapper
? include ‘status‘ as a standard post-deploy operator command

Phase 2: upgrade workflow

? add ‘mpc-infra upgrade‘
? default target should resolve from the latest published release
? support ‘--tag <tag>‘ for explicit pinning
? check the currently deployed tag before changing anything
? provide post-upgrade verification output

Phase 3: greenfield GCP bootstrap

? bootstrap a new partner GCP project from near-empty state
? enable required APIs
? create the Terraform state bucket
? validate IAM prerequisites
? optionally create baseline secrets placeholders and other required environment scaffolding

Phase 4: reduce Terraform surface area

? simplify the Terraform variable contract
? move more defaults out of user-editable tfvars
? reduce direct editing of raw Terraform inputs

Phase 5: optional deeper cleanup

? split internal-only vs partner-facing modules
? formalize profiles for mainnet/testnet
? add more robust deployment health/status commands

RISKS AND MITIGATIONS

Risk: wrapper becomes a second system to maintain

Mitigation:

? keep it thin
? treat Terraform as the source of truth for infrastructure behavior
? only own translation, validation, and user experience in the wrapper

Risk: Terraform internals still leak through

Mitigation:

? standardize generated tfvars
? do not require partners to edit Terraform files directly in the normal path

Risk: feature growth keeps adding new partner inputs

Mitigation:

? group feature-specific settings into profiles or feature blocks
? prefer team-owned defaults where possible
? add validation messages that explain what new inputs are required and why

RECOMMENDED PHASE 1 DELIVERABLES

1. A design-approved partner config schema.
2. A wrapper CLI skeleton.
3. A validation command with strong preflight checks.
4. A Terraform rendering step that outputs generated tfvars.
5. A guided deploy command.
6. Updated README flow that makes the wrapper the default path.

SUGGESTED SUCCESS METRICS

? fewer partner setup support messages
? fewer Terraform failures caused by missing secrets or APIs
? faster time from project creation to successful first deploy
? fewer docs-only onboarding questions
? fewer partner edits to raw Terraform files

RECOMMENDATION

Proceed with a **thin ‘mpc-infra‘ CLI wrapper over the current partner mainnet Terraform**
rather than a ground-up infrastructure redesign.

That gives the team the best balance of:

? lower partner overhead
? preserved GCP-only assumptions
? continued ability to change deployment details
? lower migration risk
? a gradual path away from raw Terraform as the primary partner interface

NEXT STAGES

After this design document, the recommended order is:

1. define the toolset for the ‘mpc-infra‘ implementation
2. propose repo code additions and file layout
3. draft the first code changes
4. update the README so the wrapper becomes the primary documented path